Topology Optimization
Structural optimization of 2D distributed systems

Tommaso Bocchietti 10740309

A.Y. 2024/25

POLITECNICO
MILANO 1863

Contents

List of Figures

1 Methodology

The aim of this work is to optimize the topology of 2D distributed rectangular systems, given a set of constraints and a set of loads.
In the following sections, we will at first give an overview of the topology optimization problem in case of distributed systems, describing the main concepts and the mathematical foundations needed.
Then, we will briefly present the *99 lines code* by Sigmun [2] which is a simple (yet complete) implementation of topology optimization for 2D distributed systems.
Finally, by directly modifying the code by Sigmund, we will perform different optimization tasks considering different constraints and loads, and we will analyze the results obtained.

2 Theoretical background

In this section, we will provide a brief overview of the theoretical background needed to understand the mechanics of the topology optimization problem for 2D distributed systems. In particular, we will focus on the variable thickness sheet problem, which is the main subject of this work.
For a more in-depth analysis of the topic, we suggest consulting the references provided at the end of this document.

2.1 Variable thickness sheet problem

The variable thickness sheet problem is a classical example of a topology optimization problem for 2D distributed systems. The goal is to find the optimal distribution of material within a given design domain, subject to a set of constraints and loads.
The objective function to be minimized is typically the compliance of the structure, which is a measure of its ability to deform under load. Instead, the design variables are the thickness of the sheet at each point in the domain, which can assume continuous values within a predefined range.
Under these conditions, the problem can be formulated as follows:

$$(P)_{nf} \quad \begin{cases} \min_{\boldsymbol{x}} & C(\boldsymbol{x}) = \boldsymbol{F}^T \boldsymbol{u} \\ \text{s. t.} & \sum_{j=1}^{n} V_j - V_{max} \leq 0 \\ & \boldsymbol{x} \in \Omega \end{cases} \tag{1}$$
$$\Omega := \{\boldsymbol{x} \in \mathbb{R}^n \mid 0 < x_j^{min} \leq x_j \leq 1, \quad j = 1, \dots, n\}$$

Where one can notice the definition of the compliance function $C(\boldsymbol{x})$ as for the objective function, and the constraint on the volume of the material V_j. As said before, in order to avoid singularities in the optimization process, the thickness of the sheet is forced to never assume a value of zero, hence the lower bound $0 < x_j^{min}$.

2.2 Optimality criteria method

Equation 1 is nothing more that a traditional compliance minimization problem with a volume constraint, which is a common formulation for structural optimization problems. One can show that the problem is convex, and can be solved efficiently using a variety of optimization algorithms. Among those, we recall the CONLIN (Convex Linearization) or MMA (Method of Moving Asymptotes) algorithms, which are widely used in the field of topology optimization.
However, in this work, we will focus on the Optimality Criteria (OC) method, which is a simple yet effective optimization algorithm that is particularly suitable for the variable thickness sheet problem.
In order to apply the OC method, we need at first to move to a Lagrangian formulation of the optimization problem. To do so, we can work from the compliance definition and derive its sensitivity with respect to the design variables $\boldsymbol{x}$:

$$C(\boldsymbol{x}^k) = \boldsymbol{F}^T \boldsymbol{u}^k = \boldsymbol{u}^{T,k} \boldsymbol{K}^k \boldsymbol{u}^k = \sum_{j=1}^{n} \boldsymbol{u}_j^{T,k} \boldsymbol{K}_j^k \boldsymbol{u}_j^k = \sum_{j=1}^{n} (x_j^k) \boldsymbol{u}_j^{T,k} \boldsymbol{K}_{0j}^k \boldsymbol{u}_j^k \tag{2}$$

Where $\boldsymbol{K}_{0j}^k$ is the stiffness matrix of the j-th element in the structure, and $\boldsymbol{u}_j^k$ is the displacement vector of the same element.
By differentiating Equation 15 with respect to the design variables, we can obtain the sensitivity of the compliance function:

$$\frac{\partial C}{\partial x_i} = -\boldsymbol{u}_i^{T,k} \boldsymbol{K}_{0i}^k \boldsymbol{u}_i^k \tag{3}$$

The OC method is based on the idea of approximating the compliance function with a linear function, and then updating the design variables in order to minimize the compliance. This approach is extremely similar to the one adopted in the CONLIN algorithm, but the interviewing variables are different. In particular, for the OC method, the interviewing variables are defined as:

$$y_e = x_e^{-\alpha} \quad \alpha > 0 \tag{4}$$

By doing so, the compliance function can be approximated via Taylor expansion as:

$$C(\boldsymbol{x}) \approx C(\boldsymbol{x}^k) + \sum_{j=1}^{n} \frac{\partial C}{\partial y_e}\Big|_{\boldsymbol{x}^k} (y_e - y_e^k) \tag{5}$$

One can compute the derivative of the compliance function with respect to the interviewing variables as:

$$\frac{\partial C}{\partial y_e} = \frac{\partial C}{\partial x_e}\frac{\partial x_e}{\partial y_e} = \frac{1}{\alpha}(\boldsymbol{u}_e^k)^T \boldsymbol{K}_e^0 \boldsymbol{u}_e^k (\boldsymbol{x}_e^k)^{\alpha+1} \tag{6}$$

Substituting in Equation 5, we obtain the following linear approximation of the compliance function:

$$C(\boldsymbol{x}) \approx C(\boldsymbol{x}^k) + \sum_{j=1}^{n} \frac{1}{\alpha}(\boldsymbol{u}_e^k)^T \boldsymbol{K}_e^0 \boldsymbol{u}_e^k (\boldsymbol{x}_e^k)^{\alpha+1}(y_e - y_e^k) \approx \text{const} + \sum_{j=1}^{n} b_e^k x_e^{-\alpha} \tag{7}$$

Where $b_e^k = \frac{1}{\alpha}(\boldsymbol{u}_e^k)^T \boldsymbol{K}_e^0 \boldsymbol{u}_e^k (\boldsymbol{x}_e^k)^{\alpha+1}$.
The linearized optimization problem can then be formulated as:

$$(P)_{oc} \quad \begin{cases} \min_{\boldsymbol{x}} & \sum_{j=1}^{n} b_e^k x_e^{-\alpha} \\ \text{s. t.} & \sum_{j=1}^{n} V_j - V_{max} \leq 0 \\ & \boldsymbol{x} \in \Omega \end{cases} \tag{8}$$
$$\Omega := \{\boldsymbol{x} \in \mathbb{R}^n \mid 0 < x_j^{min} < x_j \leq x_j^{max}, \quad j = 1, \ldots, n\}$$

As is commonly done, to solve this linearized optimization problem, we can adopt the Lagrangian duality formulation, which reads:

$$(D)_{oc} \quad \begin{cases} \max_{\boldsymbol{\lambda}} & \min_{\boldsymbol{x} \in \Omega} \mathcal{L}(\boldsymbol{x}, \boldsymbol{\lambda}) \\ \text{s. t.} & \boldsymbol{\lambda} \geq 0 \end{cases} \tag{9}$$
$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\lambda}) = \sum_{j=1}^{n} b_e^k x_e^{-\alpha} + \boldsymbol{\lambda}^T \left(\sum_{j=1}^{n} V_j - V_{max} \right)$$

The dual optimization problem $(D)_{oc}$ can be solved by finding the optimal values of the Lagrange multipliers $\boldsymbol{\lambda}$, which can be done by solving the following equation:

$$\frac{\partial \mathcal{L}}{\partial x_j} = -\alpha b_e^k x_e^{-\alpha-1} + \lambda_j a_e = 0 \rightarrow x_e^* = \left(\frac{\alpha b_e^k}{\lambda_j a_e} \right)^{\frac{1}{\alpha+1}} \tag{10}$$

We should always check that the solution of the dual optimization problem satisfies the constraints of the primal optimization problem, that is:

$$x_j^{(k+1)}(\lambda) = \begin{cases} x_j^* & \text{if } x_j^* \in [x_j^{min}, x_j^{max}] \\ x_j^{min} & \text{if } x_j^* < x_j^{min} \\ x_j^{max} & \text{if } x_j^* > x_j^{max} \end{cases} \tag{11}$$

Finally, we can use the following update rule to find the optimal value of the Lagrangian multiplier λ:

$$\frac{\partial \psi^k(\lambda)}{\partial \lambda} = \sum_{j=1}^{n} a_e x_j^{(k+1)} - V_{max} = 0 \tag{12}$$

So basically, one can solve for the Lagrangian multiplier λ by finding numerically the root of the equation above. Once the optimal value of λ is found, we can update the design variables $\boldsymbol{x}$ using the projection operator defined above, and then update the Lagrangian function $\mathcal{L}^k(\boldsymbol{x}, \boldsymbol{\lambda})$. This iterative procedure can be repeated until convergence is reached.

2.3 Penalizing intermediate densities

When dealing with density based topology optimization problems, it is common practice to penalize intermediate values of the design variables. This is done to encourage the optimization process to find solutions with a more binary distribution of material, which is often more suitable for manufacturing processes.
One of the most common penalization methods is the Solid Isotropic Material with Penalization (SIMP) method. In particular, the SIMP method is based on the following penalization function:

$$E(\rho) = E_0 \rho^p \tag{13}$$

Where E_0 is the Young's modulus of the material, ρ is the density of the material at a given point in the domain, and p is the penalization factor. In order to be effective, the penalization factor p should be chosen such that $p > 1$ and typically $p \approx 3$.
In Figure 1, we can see the effect of the penalization factor and how the penalization function behaves for different values of p.

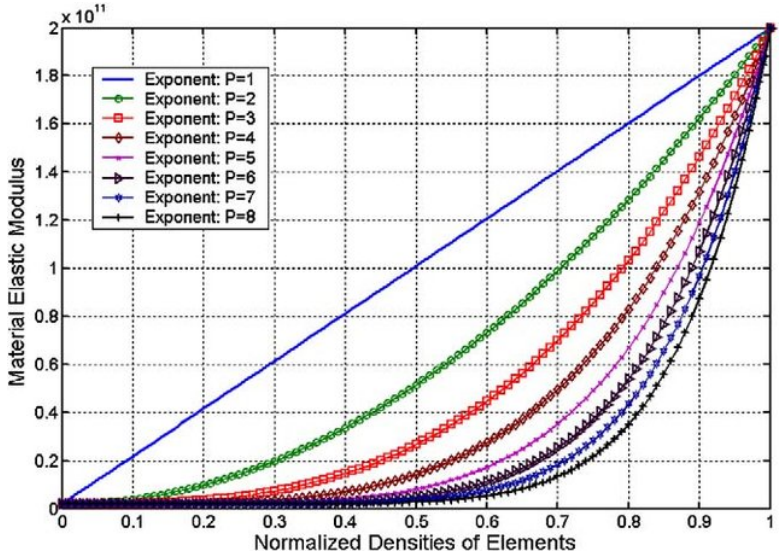

Figure 1: Effect of the penalization factor p on the SIMP penalization function.

As we can see, for $p = 1$, the penalization function is linear and does not penalize intermediate values of the density. On the other hand, for $p > 1$, the penalization function becomes more aggressive and penalizes intermediate values more severely.
Of course, by adopting the SIMP method, one should also adjust the optimization algorithm slightly. In particular, starting from the global stiffness formulation, we observe that:

$$K(\boldsymbol{x}) = \sum_{e=1}^{n} x_e^p K_e^0 \tag{14}$$

From which we can derive the sensitivity of the compliance function with respect to the design variables:

$$\frac{\partial C}{\partial x_e} = -\boldsymbol{u}^T (p\rho_e^{p-1} K_e^0)\boldsymbol{u} \tag{15}$$

Because of this new formulation for the sensitivity of the compliance function, we also have to adjust the update rule for the design variables in the optimization algorithm. In particular, recalling the criteria for the optimality of the design variables in Equation **??**, the previously defined b_e^k term should be updated as follows:

$$b_e^k = -\frac{\partial C}{\partial y_e} = \boldsymbol{u}^T (p\rho_e^{p-1} K_e^0)\boldsymbol{u} \tag{16}$$

The rest of the optimization algorithm remains unchanged, and the optimization process can proceed as usual.

2.4 Sensitivity filtering

One of the most severe issues in topology optimization is the checkerboard pattern, which is a common artifact that appears in the optimized designs. In order to mitigate this issue, it is common practice to apply a sensitivity filtering technique to the optimization algorithm.

In particular, a commonly used technique to reduce the risk of checkerboard patterns is the so-called sensitivity filtering, in which the sensitivities are convoluted with a filter kernel. In other words, the density of each element is updated based on the average sensitivities of its neighboring elements, rather than the sensitivities of the element itself.
Such a filter, can be defined over the compliance sensitivities as:

$$\frac{\partial \hat{C}}{\partial x_j} = \frac{1}{x_j \sum_j H_j} \sum_j H_j x_j \frac{\partial C}{\partial x_j} \tag{17}$$

Where H_j is the filter kernel, and the sum is performed over the neighboring elements of the element j. The filter kernel H_j can be defined in different ways, but a common choice is the following:

$$H_j = r_{min} - dist(j,i) \quad \{i \in N \mid dist(j,i) \leq r_{min}\} \tag{18}$$

3 99 lines code

All the previously described theoretical foundation has been already implemented by Sigmund [2] in it's 99 lines code. The code is a `MATLAB` function that solves the compliance minimization problem for a 2D distributed rectangular domain with a uniform grid of elements. Penalization of intermediate densities is achieved base on SIMP model and a simple anti-checkerboard filter is applied to avoid checkerboard patterns. The method used to solve the optimization problem is the method of moving Optimality Criteria (OC).
In Appendix A the flowchart for a generic OC algorithm is shown which is the same used in the 99 lines code. The *99 lines code* can be found online at this link.

Parameters Before jumping to the requests and solutions for the given problem, let's first introduce the parameters that can be set in the 99 lines code:

```
function x = top(nelx, nely, volfrac, penal, rmin)
    % Rest of the code ...
end
```

From the function signature we can see that the function `top` takes 5 parameters:

- `nelx`: number of elements in the x-direction
- `nely`: number of elements in the y-direction
- `volfrac`: volume fraction (α)
- `penal`: penalization factor (p)
- `rmin`: filter radius (r_{min})

4 Requests and solutions

In the following, the requests and their solutions for this project are presented.

4.1 Case 1: different p and r_{min} values

For this case, the considered structure is the one depicted in Figure 2.

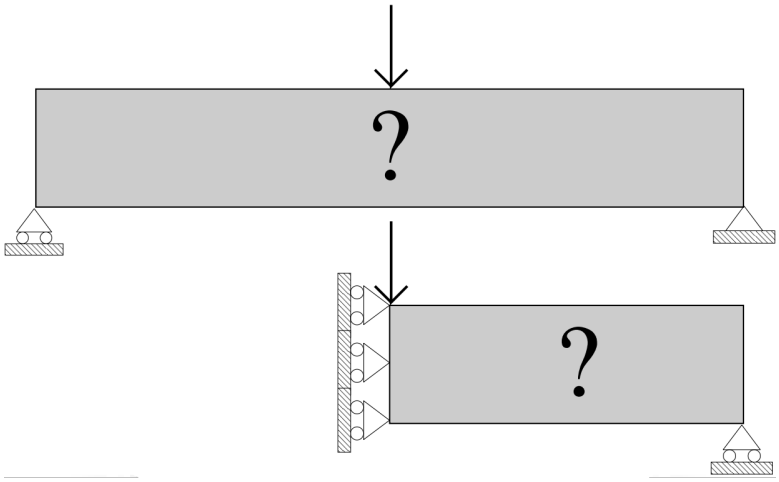

Figure 2: Initial structure for Case 1

In Figure 3, the results of the simulation for different values of p and r_{min} are shown.

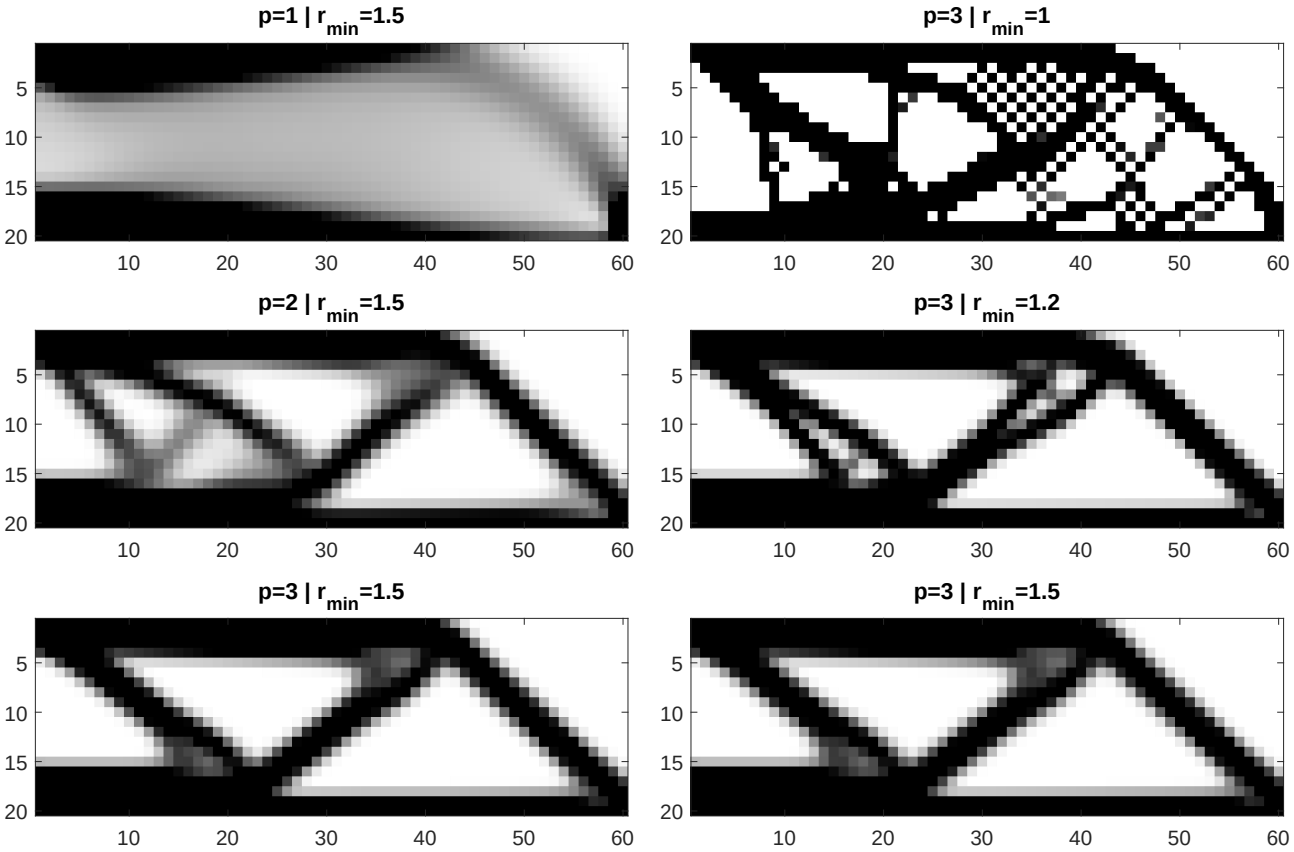

Figure 3: Case 1: different p and r_{min} values.

One can clearly see how the two parameters impact the final results.
The higher the p value, the more pronounced the penalization of intermediate densities values. By doing so, the algorithm is forced to find a solution with a more binary distribution of the material which is in general beneficial for a possible manufacturing process.
The r_{min} parameter, which is the one controlling the filter radius, has also a significant impact on the final results. For smaller values of r_{min} the checkerboard pattern is clearly visible.

4.2 Case 2: different boundary conditions

For this case, the considered structure is the one depicted in Figure 4.

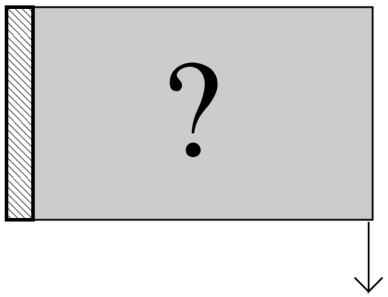

Figure 4: Initial structure for Case 2

In order to consider the different boundary conditions, the following lines of code were modified:

```
% DEFINE LOADS AND SUPPORTS (HALF MBB-BEAM)
F(2*(nelx+1)*(nely+1), 1) = -1;
fixeddofs = 1:2*(nely+1);
```

The optimized structure for the given loads and boundary conditions is shown in Figure 5.

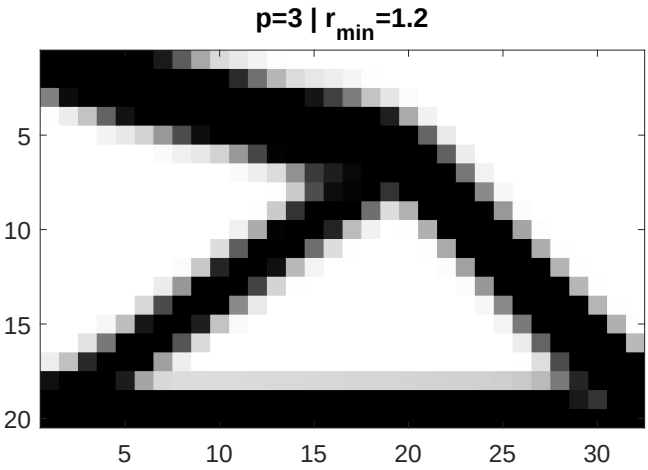

Figure 5: Case 2: different boundary conditions.

4.3 Case 3: multiple load cases

For this case, the considered structure is the one depicted in Figure 6.

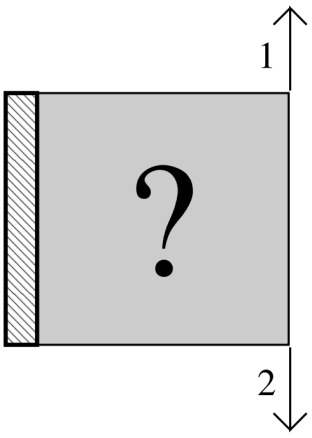

Figure 6: Initial structure for Case 3

Given the request, we now consider two different situations: one with the two loads applied simultaneously and one with the two loads applied separately.
When the loads are applied simultaneously, the following lines of code are used:

```
% DEFINE LOADS AND SUPPORTS (HALF MBB-BEAM)
F(2*(nelx+1)*(nely+1), 1) = -1;
F(2*(nelx)*(nely+1)+2, 1) = +1;
fixeddofs = 1:2*(nely+1);
```

Instead, when the loads are applied separately, the following lines of code are used:

```
% Inside the Sensitivity Analysis computation
for i = 1:2
    Ue = U([ 2*n1-1; 2*n1; 2*n2-1; 2*n2; 2*n2+1; 2*n2+2; 2*n1+1; 2*n1+2 ], i);
    c = c + x(ely,elx)^p*Ue'*KE*Ue;
    dc(ely,elx) = -p*x(ely,elx)^(p-1)*Ue'*KE*Ue;
end

% Inside the FE function
F = sparse(2*(nely+1)*(nelx+1), 2);
U = sparse(2*(nely+1)*(nelx+1), 2);

% DEFINE LOADS AND SUPPORTS (HALF MBB-BEAM)
F(2*(nelx+1)*(nely+1), 1) = -1;
F(2*(nelx)*(nely+1)+2, 2) = +1;
```

The optimized structures for the two different situations are shown in Figure 7.

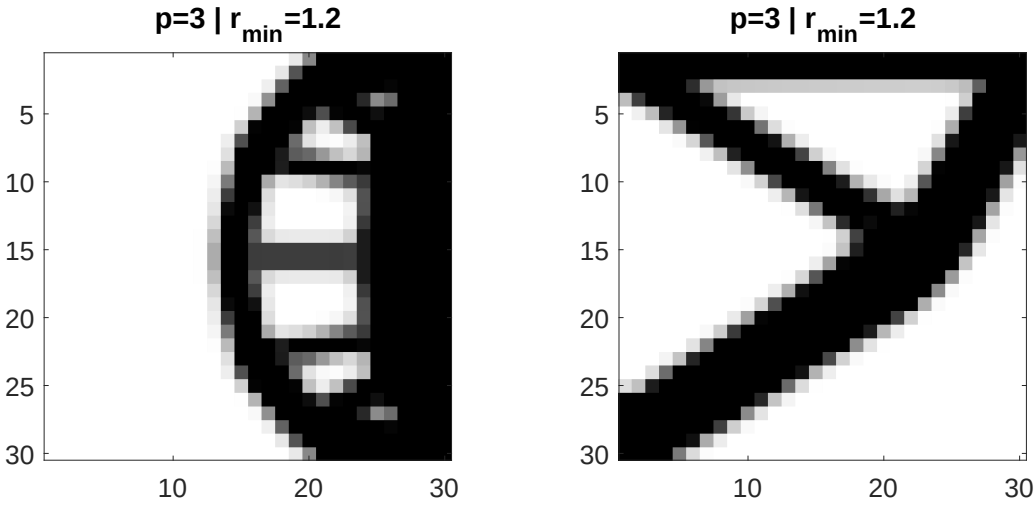

Figure 7: Case 3: multiple load cases.

As we can see, the two different load situations lead to different optimized structures.
In particular, when the loads are applied simultaneously, the reaction forces are null and the structure is optimized as if there were no connection to the ground. This also explains why the density of the structure is almost null in the left-hand side of the structure.
On the other hand, when the loads are applied separately, the reaction forces are not null and the structure is optimized accordingly.

4.4 Case 4: passive elements

For this case, the considered structure is the one depicted in Figure 8.

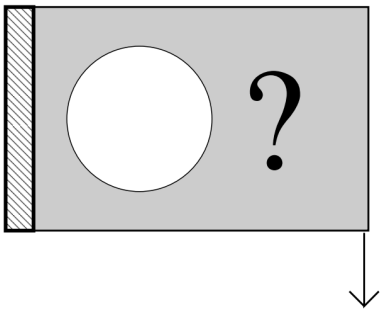

Figure 8: Initial structure for Case 4

To solve this problem we generate an array that defines which elements must be preserved. This array should act as a map that restores the properties to be preserved for the required elements.
The following lines of code were added to the main script:

```
% Before the main optimization loop starts
passive = zeros(nely, nelx);
for ely = 1:nely
    for elx = 1:nelx
        if sqrt((ely-nely/2.)^2+(elx-nelx/3.)^2) < nely/3.
            passive(ely,elx) = 1;
            x(ely,elx) = 0.001;
        end
    end
end

% Inside the Optimality Criteria loop
xnew(passive == 1) = 0.001;
```

The optimized structure for this case is shown in Figure 9.

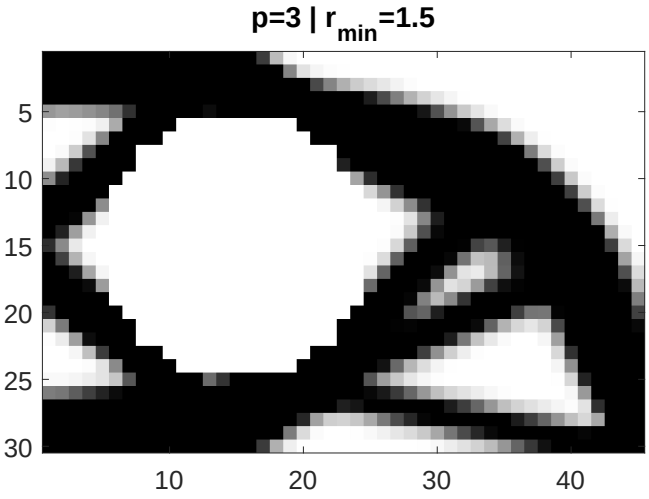

Figure 9: Case 4: passive elements.

References

[1] Anders Klarbring Peter W. Christensen. *An Introduction to Structural Optimization*. Springer Dordrecht, 2010.

[2] O. Sigmund. A 99 line topology optimization code written in matlab. *Structural and Multidisciplinary Optimization*, 21(2):120–127, Apr 2001.

A Density-based topology optimization

The flowchart in Figure 10 shows the steps to solve a generic density-based topology optimization problem. One can refer to Section 2 for a detailed explanation of the theoretical background.

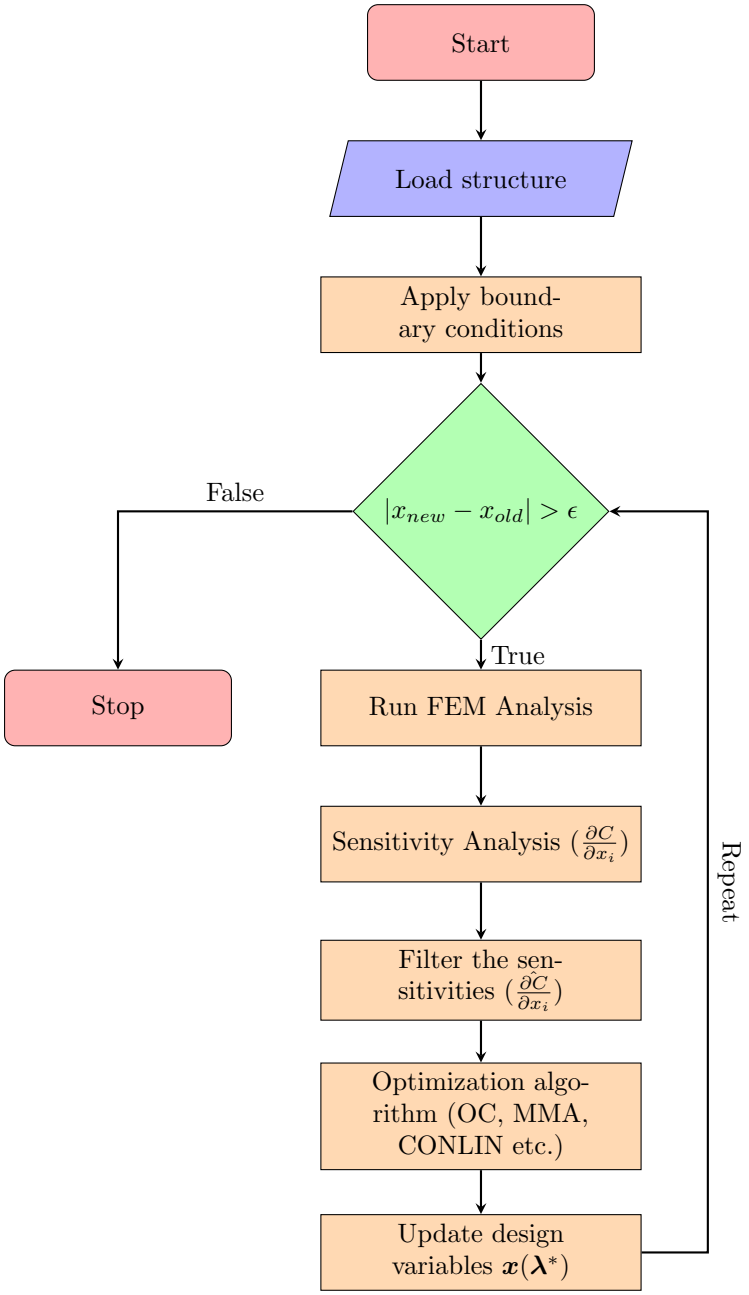

Figure 10: Compliance minimization flowchart